

PARCIAL PRÁCTICO CORTE I

Adquisición de datos ambientales con ESP32

Integrantes

Andrea Fonseca Gómez

Juan Cupaban Mejia

Grupo

B192

1. Identificación del problema

El aula requiere un sistema de monitoreo capaz de obtener información ambiental y de presencia a partir de sensores electrónicos. Para esta práctica se plantea una solución con ESP32 que lea temperatura, humedad, nivel de iluminación y distancia, y que muestre los datos en el Monitor Serial.

La actividad del parcial define como problema ABPr que el sistema de monitoreo del aula necesita reportar temperatura, humedad, nivel de luz y detectar presencia mediante distancia. La presencia se determina cuando la distancia medida es menor a 30 cm. [1]

2. Justificación

La adquisición simultánea de variables ambientales permite integrar en un solo prototipo diferentes tipos de entradas: una entrada digital para temperatura y humedad, una entrada analógica para iluminación y una entrada por temporización de pulsos para distancia. El ESP32 concentra estas lecturas y las presenta de forma estructurada en el Monitor Serial, lo que facilita la comprobación del funcionamiento y constituye una base para una futura solución IoT conectada por Wi-Fi.

La solución también permite demostrar la relación entre sensores, procesamiento y visualización, que es el objetivo central del ejercicio práctico. En lugar de asumir mediciones reales antes de montar el circuito, el presente informe documenta el diseño y los valores esperados; las lecturas definitivas deben comprobarse con el prototipo físico.

3. Objetivos

Objetivo general

Implementar un prototipo con ESP32 capaz de adquirir temperatura, humedad, nivel de luz y distancia, y visualizar dichas variables en el Monitor Serial para apoyar el monitoreo básico de un aula.

Objetivos específicos

- Conectar y leer un sensor DHT11 para obtener temperatura y humedad.
- Construir un divisor de tensión con una LDR y una resistencia de 10 k Ω y leerlo mediante el ADC del ESP32 en GPIO34.
- Integrar un sensor HY-SRF05 como sustituto funcional del HC-SR04 solicitado en la guía, utilizando TRIG y ECHO separados.
- Aplicar una condición lógica que identifique el estado Ocupado cuando la distancia sea menor de 30 cm.
- Unificar las lecturas en un solo programa Arduino IDE, con comentarios por sección y salida organizada en el Monitor Serial.
- Documentar el montaje electrónico, la arquitectura del prototipo y las características básicas del sensor MQ-2.

4. Diagrama de arquitectura IoT

La arquitectura se organiza en tres funciones: adquisición de datos mediante los sensores, procesamiento local en el ESP32 y visualización mediante el Monitor Serial. El envío a una plataforma IoT por Wi-Fi se muestra únicamente como una extensión futura, porque no hace parte de la práctica base descrita en el parcial. [1]



Figura 1. Arquitectura funcional del prototipo. Elaboración propia con base en los requerimientos del parcial.

5. Lista de materiales

Material	Cantidad	Función
ESP32 DevKit	1	Controlador y adquisición de datos
DHT11	1	Temperatura y humedad
LDR / fotorresistencia	1	Nivel de iluminación
Resistencia 10 kΩ	1	Divisor de tensión de la LDR
HY-SRF05	1	Medición de distancia y presencia
Resistencia 1 kΩ	1	Divisor de tensión para ECHO
Resistencia 2 kΩ	1	Divisor de tensión para ECHO
Resistencia 4.7-10 kΩ	1 opcional	Pull-up si el DHT11 es el sensor desnudo de 4 pines

Protoboard	1	Montaje
Cables jumper	Varios	Interconexión
Cable USB / fuente 5 V	1	Alimentación y programación

6. Diagrama electrónico y conexiones

El circuito utiliza 3.3 V para el DHT11 y el divisor de la LDR. El HY-SRF05 se alimenta a 5 V en su pin VCC. Como la salida ECHO del HY-SRF05 es una señal TTL que puede elevarse hasta VCC, se incorpora un divisor de 1 kΩ y 2 kΩ antes del GPIO18. Espressif indica que la tolerancia de los GPIO del ESP32 es 3.6 V y recomienda utilizar un divisor cuando una señal supere ese nivel. [2][3]

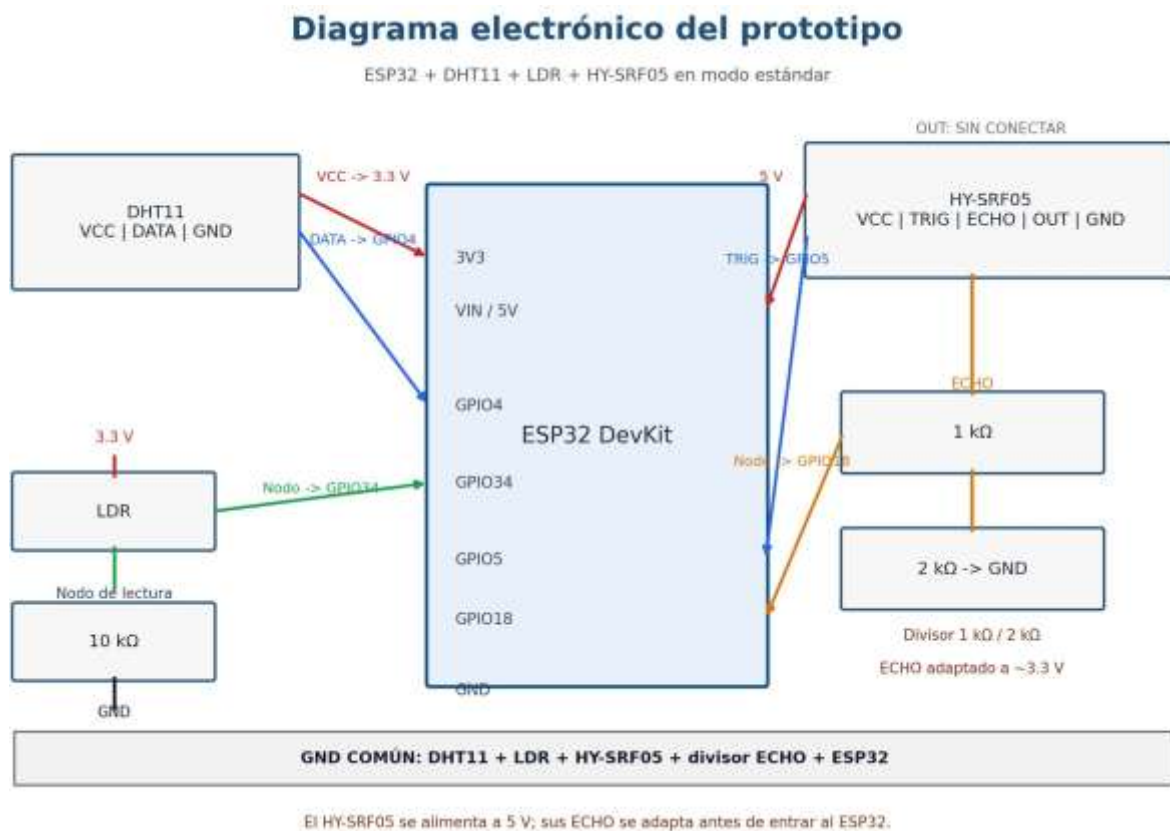


Figura 2. Diagrama electrónico propuesto. Elaboración propia.

Tabla de conexión pin por pin

Componente	Pin	ESP32 / conexión	Observación
DHT11	VCC	3.3 V	Alimentación
DHT11	DATA	GPIO4	Entrada digital
DHT11	GND	GND	Tierra común

LDR	Pata 1	3.3 V	Divisor
LDR	Pata 2	Nodo -> GPIO34	Punto de lectura
10 kΩ	Pata 1	Nodo de GPIO34	Divisor
10 kΩ	Pata 2	GND	Divisor
HY-SRF05	VCC	VIN / 5V	Según serigrafía de la placa
HY-SRF05	TRIG	GPIO5	Salida del ESP32
HY-SRF05	ECHO	1 kΩ -> nodo -> GPIO18	No conectar directo al ESP32
HY-SRF05	OUT	Sin conexión	Modo estándar TRIG/ECHO
HY-SRF05	GND	GND	Tierra común
ECHO divisor	1 kΩ	ECHO -> nodo	Serie
ECHO divisor	2 kΩ	Nodo -> GND	Aproxima ECHO a 3.3 V

Cálculo del divisor de tensión de ECHO

Con una resistencia de 1 kΩ en la parte superior y una resistencia de 2 kΩ a GND, la tensión aproximada en el GPIO18 cuando ECHO está en 5 V es:

$$V_{out} = V_{in} \times R2 / (R1 + R2) = 5 \text{ V} \times 2 \text{ k}\Omega / (1 \text{ k}\Omega + 2 \text{ k}\Omega) \approx 3.33 \text{ V}$$

Este valor queda por debajo de la tolerancia de 3.6 V indicada por Espressif. [2]

7. Código integrado para Arduino IDE

El siguiente programa integra las tres lecturas solicitadas. Para la presencia se aplica la condición del parcial: distancia menor de 30 cm = Ocupado. El código no afirma que las lecturas sean exactas antes de realizar la prueba física; deben verificarse en el prototipo.

```
#include <DHT.h>
```

```
// =====
```

```
// DHT11
```

```
// =====
```

```
#define DHTPIN 13
```

```
#define DHTTYPE DHT11
```

```
DHT dht(DHTPIN, DHTTYPE);
```

```

// =====
// LDR
// =====

#define LDR_PIN 4

// =====

// HY-SRF05
// =====

#define TRIG_PIN 5
#define ECHO_PIN 18

// =====

// CONFIGURACIÓN
// =====

void setup() {

    Serial.begin(115200);

    dht.begin();

    pinMode(LDR_PIN, INPUT);

    pinMode(TRIG_PIN, OUTPUT);
    pinMode(ECHO_PIN, INPUT);

    digitalWrite(TRIG_PIN, LOW);

    delay(2000);

    Serial.println("=====");
    Serial.println("    MONITOREO AULA");
    Serial.println("=====");

```

```

}

// =====
// MEDIR DISTANCIA
// =====

float medirDistancia() {

    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);

    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);

    digitalWrite(TRIG_PIN, LOW);

    long duracion = pulseIn(ECHO_PIN, HIGH, 30000);

    if (duracion == 0) {
        return -1;
    }

    float distancia = duracion * 0.0343 / 2.0;

    return distancia;
}

// =====
// PROGRAMA PRINCIPAL
// =====

void loop() {

    // -----

```

```

// DHT11
// -----
float temperatura = dht.readTemperature();
float humedad = dht.readHumidity();

// -----
// LDR
// -----
int luz = digitalRead(LDR_PIN);

// -----
// HY-SRF05
// -----
float distancia = medirDistancia();

// -----
// MOSTRAR INFORMACIÓN
// -----
Serial.println();
Serial.println("=== MONITOREO AULA ===");

if (isnan(temperatura)) {

    Serial.println("Temperatura: Error");

} else {

    Serial.print("Temperatura: ");
    Serial.print(temperatura, 1);
    Serial.println(" °C");
}

```

```
if (isnan(humedad)) {

    Serial.println("Humedad: Error");

} else {

    Serial.print("Humedad: ");
    Serial.print(humedad, 1);
    Serial.println(" %");
}

if (luz == HIGH) {

    Serial.println("Luz: Baja");

} else {

    Serial.println("Luz: Detectada");
}

if (distancia < 0) {

    Serial.println("Distancia: Sin lectura");

} else {

    Serial.print("Distancia: ");
    Serial.print(distancia, 1);
    Serial.println(" cm");

    // -----
    // DETECCIÓN DE PRESENCIA
```

```
// -----
```

```
if (distancia < 30.0) {
```

```
    Serial.println("Estado: Ocupado (distancia < 30 cm)");
```

```
} else {
```

```
    Serial.println("Estado: Libre (distancia >= 30 cm)");
```

```
}
```

```
}
```

```
delay(1000);
```

```
}
```

8. Formato de salida y criterios de prueba

El parcial define el formato general de salida mostrado a continuación y establece la condición de presencia a menos de 30 cm. [1]

```
== MONITOREO AULA ==  
Temperatura: 24.5 °C  
Humedad: 62.3 %  
Luz: Detectada  
Distancia: 15.2 cm  
Estado: Ocupado (distancia < 30 cm)
```

9. Investigación: sensor MQ-2 para calidad del aire

¿Qué gases detecta?

El MQ-2 es un sensor semiconductor orientado a la detección de gases combustibles y humo. La hoja de datos del sensor especifica una concentración de detección de 300 a 10 000 ppm para gases combustibles y caracteriza el sensor para aplicaciones como detectores portátiles de gas. [5]

En módulos comerciales se utiliza para reaccionar ante gases combustibles y humo; entre los gases asociados con su sensibilidad se encuentran LPG, propano, metano e hidrógeno. La salida no debe interpretarse automáticamente como una concentración exacta de un gas específico sin calibración y condiciones de prueba controladas.

¿Cuál es su tiempo de precalentamiento?

La hoja de datos consultada especifica un tiempo de precalentamiento de más de 48 horas para obtener condiciones estabilizadas de funcionamiento. Para una demostración inicial, algunos módulos comerciales pueden reaccionar antes, pero esa respuesta temprana no debe confundirse con el tiempo de estabilización indicado por la documentación del sensor. [5]

¿Cómo se conecta al ESP32?

Un módulo MQ-2 suele exponer VCC, GND, AOUT y DOUT. Como el elemento calefactor trabaja a 5 V y la salida analógica del módulo puede aproximarse a su alimentación, una integración segura con el ESP32 debe adaptar el nivel de AOUT mediante un divisor antes de un ADC. Una conexión de referencia es: VCC -> 5 V/VIN; GND -> GND; AOUT -> divisor -> GPIO34; DOUT -> un GPIO digital opcional mediante una conexión compatible con niveles lógicos.

Ejemplo de divisor para AOUT del MQ-2:

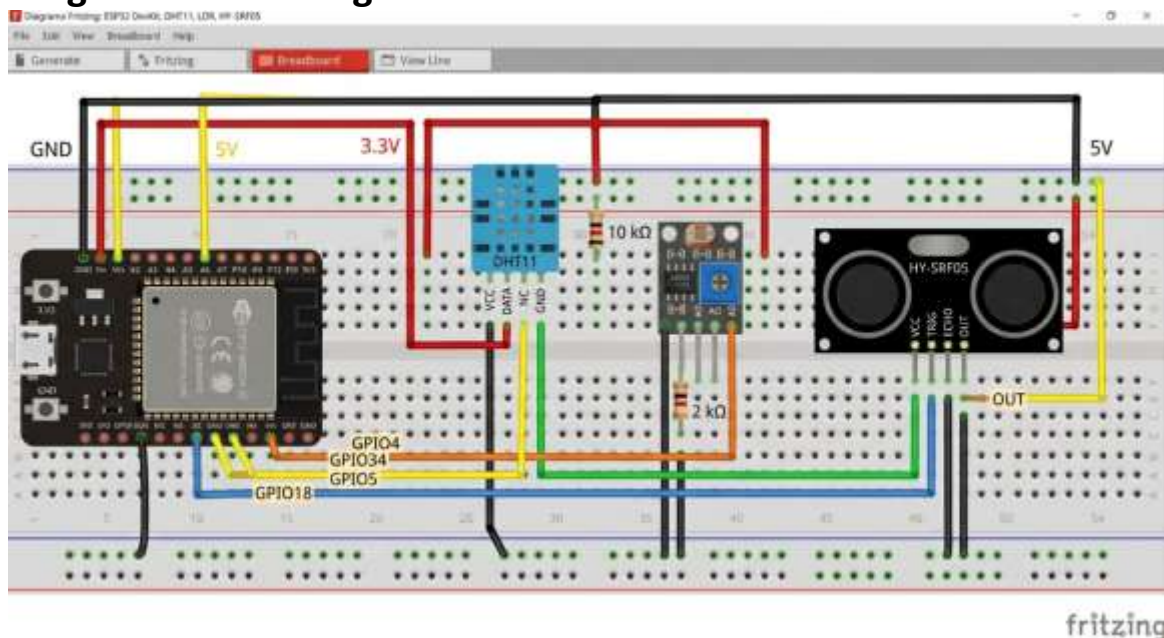
MQ-2 AOUT ---- 10 kΩ ----+---- GPIO34
|
20 kΩ
|
GND

Con 5 V de entrada al divisor:

$V_{out} = 5 \text{ V} \times 20 / (10 + 20) \approx 3.33 \text{ V}$

Esta conexión se presenta como una propuesta de adaptación de nivel, no como una prueba realizada durante este parcial. La integración del MQ-2 queda documentada porque es una actividad específica del enunciado, mientras que el código principal se mantiene con los tres sensores exigidos para la práctica integrada. [1]

10. Diagrama en Fritzing



11. Cronograma de desarrollo propuesto

Semana	Actividad	Producto / evidencia
1	Definición del problema y objetivos	Planteamiento y objetivos
2	Montaje DHT11 y LDR	Pruebas de temperatura, humedad y luz
3	Montaje ultrasónico	Pruebas de distancia y condición de presencia
4	Integración del código	Programa multisensor comentado
5	Diseño y documentación	Arquitectura, circuito e investigación MQ-2
6	Pruebas finales y entrega	Informe, video, código y prototipo funcional

Nota: cronograma propuesto para organizar las actividades del entregable; el enunciado solicita incluir un cronograma, pero no define fechas intermedias. [1]

12. Referencias técnicas

- [1] PARCIAL PRÁCTICO Corte I (semana 6), “Adquisición de datos ambientales con ESP32”. Documento suministrado para la práctica. El enunciado establece los materiales, procedimiento, formato de salida, condición de presencia, diagrama, investigación MQ-2 y entregables.
- [2] Espressif Systems. ESP-FAQ - Hardware related. Documentación oficial sobre GPIO34-GPIO39 como entradas y tolerancia de GPIO de 3.6 V. <https://docs.espressif.com/projects/esp-faq/en/latest/hardware-related/hardware-design.html>
- [3] Sigma NOrteč. Ultrasonic Sensor HY-SRF05 2-450 cm 5 V compatible with HC-SR04. Especificaciones de alimentación, pines, TRIG/ECHO, modos y divisor recomendado para ESP32. <https://sigmanortec.ro/en/ultrasonic-sensor-hy-srf05-5-pins-2450-cm-5v>
- [4] Adafruit Learning System. DHT11, DHT22 and AM2302 Sensors. Guía de conexión y uso de la librería DHT; también describe el pull-up para DHT11 sin módulo. <https://learn.adafruit.com/dht>
- [5] Hanwei / documentación técnica reproducida por Pololu. MQ-2 Semiconductor Sensor for Combustible Gas. Especifica detección de gas combustible y humo, rango de 300-10000 ppm y precalentamiento superior a 48 horas. https://www.pololu.com/file/download/MQ2.pdf?file_id=0J309